

Optimization of Leukemia Classification and Grading using Multi-Task Deep Learning

1. Introduction (0:00 - 2:00)

"Good morning/afternoon everyone. Today, I am presenting my major project: **Leukemia Classification and Grading using Multi-Task Deep Learning with Particle Swarm Optimization**."

The core problem I am addressing is the limitation of traditional diagnostics. Usually, automated systems only answer one question: "Is it cancer?" But in a clinical setting, we need to know the **Type** (ALL, AML, CLL, CML) and the **Grade** (Chronic vs. Acute/Blast) simultaneously.

My solution is a unified pipeline that automates this. I have implemented a **Multi-Task Learning (MTL)** system that predicts both attributes at once, optimized using **Particle Swarm Optimization (PSO)**. I will walk you through the code implementation, stage by stage."

2. Stage 1: Data Engineering & Preprocessing (2:00 - 4:00)

"The first critical stage is the data pipeline, which is handled by the `prepare_data()` function in our `final_pipeline.py`.

Function: `prepare_data()`

This is our master data handler. It doesn't just load images; it ensures consistency across the dataset.

- * **Input Handling:** It scans our raw data directories using `glob` to find all image formats (jpg, png, tif).
- * **Preprocessing:** Inside the loop, we use **OpenCV (`cv2.resize`)** to standardize every single image to **224x224 pixels**. This is crucial for our ResNet backbone.
- * **Stratified Splitting:** The function then performs a randomized split: **70% Training, 15% Validation, 15% Testing** and physically moves these files into a structured directory tree using `shutil.copy2`. This ensures that our model never sees the test data during training.

Class: `MultiTaskDataset` (inherits from `datasets.ImageFolder`)

This is arguably the most important logic in the data phase.

- * **Function: `__getitem__(index)`:** Standard datasets only return one label. I overrode this function to return **three items**: the image, the `type_label`, and the `grade_label`.
- * **Logic:** inside this function, I implemented the medical logic:
 - * If the class is **ALL** or **AML**, it assigns a grade of **Blast** (Acute).
 - * If the class is **CLL** or **CML**, it assigns a grade of **Chronic**.
- * This allows the single dataset object to feed ground truth for *both* tasks simultaneously."

3. Stage 2: Model Architecture (4:00 - 5:30)

"Moving on to the intelligence of the system. We defined a custom PyTorch class called `MultiTaskModel`.

Class: `MultiTaskModel`

- **Function: `__init__()`:** Here, we load a pre-trained **ResNet18** as our backbone.
 - Critical Step: `self.backbone.fc = nn.Identity()`. I effectively chopped off the original classification head of ResNet used for ImageNet.
 - I replaced it with two parallel linear layers: `self.head_type` (for the 4 classes) and `self.head_grade` (for the 3 grades).
 - **Function: `forward(x)`:** This defines the data flow. The image `x` passes through the shared backbone to extract feature vectors. These vectors are then sent simultaneously to both heads, returning a tuple: `(type_logits, grade_logits)`."
-

4. Stage 3: Swarm Optimization (PSO) (5:30 - 7:30)

"Before full training, we need to find the perfect hyperparameters. Instead of manual guessing, I used the `pyswarm` library.

Function: `objective_function(hyperparameters)`

This is the fitness function that the swarm optimizes. * **Inputs:** It takes a candidate `learning_rate` and `batch_size` generated by the swarm. * **Process:** It dynamically instantiates a new `MultiTaskModel` and a `DataLoader` with that specific batch size. It runs a short "tuning" training session (5 epochs). * **Output:** It returns the `avg_val_loss`. The swarm uses this value to "fly" towards better parameter combinations.

Execution

In the main block, we call `pso(objective_function, LB, UB)`. * **Result:** The logs show the swarm successfully converged on a **Learning Rate of ~0.00016** and a **Batch Size of 32**, which we then used for the final training."

5. Stage 4: Training & Loss Calculation (7:30 - 9:00)

"With the optimal parameters found, we execute the final training using `train_multitask_model()`.

Function: `train_multitask_model()`

This function manages the training loop over 100 epochs. * **The Loss Logic:** Inside the training loop, we calculate the total loss as: `loss = type_criterion(...) + 0.5 * grade_criterion(...)` I weighted the grading loss at 0.5. This tells the network: "Focus primarily on getting the Leukemia type right, but learn the grade as a supporting feature." * **Checkpointing:** We monitor validation accuracy. The code saves the model state to `multitask_model.pt` *only* when the accuracy improves, ensuring we keep the best version."

6. Stage 5: Deployment (9:00 - 10:00)

"Finally, we deployed this model using a Flask application (`main.py`).

Function: `predict_leukemia(image_path)`

This is our inference engine in the backend. * It loads the saved `multitask_model.pt`. * It preprocesses the user's uploaded image (Resize -> ToTensor -> Normalize). * It runs a `torch.no_grad()` forward pass to get the logits, applies `argmax` to find the winning class and grade, and maps them back to human-readable strings like `~AML` and `~Blast`.

Conclusion

By breaking the problem down into these specific functional components—"Data Prep, Multi-Task Modeling, PSO Optimization, and Flask Deployment"—we have created a system that is not only accurate (reaching ~97%) but also clinically relevant."